

산학협력프로젝트 수행계획서

프로젝트명: 벡터 증강 분석 쿼리를 위한 Cascaded Vector Similarity Decomposition 연구

팀명: 속도는벡터

팀원: 박세은(팀원 1, 팀장), 강재현(팀원 2), 조현빈(팀원 3), 이동욱(팀원 4)

지도교수: 박광현

상태: 초안 — 3/31 팀 논의 후 확정 → 4/1 작성 마무리 → 4/2 제출

팀 논의 사항 (연구제안서 A/B 외 수행계획서 고유 3건)

1. 역할 분담 — 누가 뭘 맡을 것인가?

현재 "전원 참여"로 되어 있는데, 실제로는 구체적 분담이 필요할 수 있음: - Exqutor 패치 빌드 + PostgreSQL 소스 컴파일 / TPC-H 데이터 생성 + 벡터 매핑 스크립트 - EXPLAIN JSON 파싱 + 시각화 Python 스크립트 / 보고서·발표자료 총괄

→ 환경 구축 시작하면서 정해도 되지만, 제출 양식이 역할 구분을 요구할 수 있으므로 **논의 필요**

2. 데이터셋 확보 — 4/7 데드라인

환경 구축 기한 4/14에서 역산하면, 대학원생한테 빌드 환경/데이터셋을 받을 수 있는지 4/7까지 확인 필요. 누가 연락할지, 어떤 형태(Docker / 바이너리 / 패치 가이드)로 요청할지, 못 받으면 자체 빌드 또는 ECQO 비교 제외로 축소할지. → **결정 필요**

3. 하드웨어/서버 — 어디서 실험할 것인가?

SF10이면 개인 머신으로 가능하지만 SF10 확대 시 서버 필수. 교수님께 서버 사용 요청 시점을 정하고, 최소한 "예상 환경: RAM 16GB+, SSD" 정도는 적어두는 게 좋겠음. → **논의 필요** (4.1절에 예상 하드웨어 추가 완료)

1. 프로젝트 개요

항목	내용
프로젝트명	벡터 증강 분석 쿼리(VAQ)를 위한 Cascaded Vector Similarity Decomposition 연구
팀명	속도는벡터
참여 인원	4명
지도교수	박광현
수행 기간	2026년 3월 ~ 6월 (1학기)

2. 연구 배경 및 목적

2.1 연구 배경

벡터 증강 분석 쿼리(VAQ)는 벡터 유사도 검색과 관계형 분석을 결합하는 차세대 데이터 분석의 핵심 쿼리 유형이다. RAG 파이프라인, 추천 시스템, 멀티모달 검색 등에서 광범위하게 사용되고 있으나, 기존 RDBMS 쿼리 옵티마이저는 벡터 조건의 선택도(selectivity)를 정확히 추정하지 못하여 잘못된 실행 계획을 생성한다.

Exqutor(arXiv:2512.09695v2, 2025)는 ECQO와 Adaptive Sampling이라는 두 기법으로 카디널리티 추정 문제를 해결하여 pgvector에서 최대 1,000배, VBASE에서 10,000배의 성능 향상을 달성하였다. 그러나 Exqutor는 벡터 조건의 구조 자체를 변환하여 성능을 개선하는 접근은 탐구하지 않았으며, VAQ의 플래닝 시간과 실행 시간을 분리하여 병목을 분석하지도 않았다.

2.2 연구 목적

본 프로젝트는 벡터 유사도 조건을 느슨한 단계와 엄격한 단계로 분해하는 **Cascaded Vector Similarity Decomposition**을 제안하고, 이 접근이 VAQ의 쿼리 성능을 개선하는지 실험적으로 검증한다. 핵심 아이디어는 Stage 1의 D_loose를 pgvector 기본 추정치(33.3%)에 실제 선택도가 근접하도록 설정하여 추정 오차를 구조적으로 최소화하고, Stage 2는 Materialized CTE 스캔으로 벡터 추정 자체를 불필요하게 만드는 것이다.

구체적 목표는 다음과 같다:

1. VAQ에서 Planning Time vs Execution Time 병목 구조를 정량적으로 프로파일링
2. Cascaded Decomposition이 쿼리 성능(end-to-end latency)에 미치는 영향 측정
3. Exqutor ECQO와의 상호보완성/대체성을 실험적으로 판별

3. 연구 질문 및 가설

RQ1 — 병목 프로파일링

VAQ에서 전체 지연 중 플래닝 시간과 실행 시간이 각각 어느 정도를 차지하는가? 벡터 조건의 선택도와 필터 복잡도에 따라 이 비중은 어떻게 변하는가?

이 단계의 결과에 따라 2단계 실험의 초점이 결정된다. 플래닝이 유의미한 병목이면 "계획 품질 개선 + 실행 가속" 양쪽을, 그렇지 않으면 "구조적 pre-filtering을 통한 실행 가속"에 집중한다.

RQ2 — 분해의 효과

단일 벡터 유사도 조건을 두 단계(느슨 → 엄격)로 분해하면 쿼리 성능(end-to-end latency)이 개선되는가? Materialized CTE의 실체화 오버헤드가 분해의 이점을 상쇄하는가?

RQ3 — ECQO 상호작용

Cascade와 Exqutor ECQO는 보완적인가, 대체적인가? ECQO가 정확한 카디널리티를 제공한 상태에서도 Cascade 분해가 추가적 성능 향상을 가져오는가?

4. 수행 방법

4.1 실험 환경

구성 요소	버전/설정
PostgreSQL	16 (Exqutor 패치 적용 빌드)
pgvector	0.7.x (논문 재현용) + 0.8.0 (비교용)
pg_hint_plan	최신 (oracle plan 확인용)
Python	3.10+ (벤치마크 스크립트, 결과 분석)
벤치마크	TPC-H VAQ (Exqutor GitHub 제공)
하드웨어 (예상)	CPU 4코어+, RAM 16GB+, SSD 저장장치

TPC-H SF1(~1GB) + SIFT1M(~512MB) 합산 약 1.5GB 규모는 개인 머신으로 충분하며, SF10 확대 시 연구실 서버 사용을 교수님께 요청할 예정이다.

4.2 데이터셋

자체 확보 가능 (확정): - **SIFT1M** (128d, 100만 벡터): ANN 벤치마크 표준 데이터셋, 공개 다운로드 가능 - **TPC-H SF1** (~1GB): 표준 분석 벤치마크, dbgen으로 자체 생성 - **sentence-transformers 임베딩** (768d): 오픈소스 모델로 로컬 생성 가능. 고차원 확장 실험(3단계) 시 사용 후보이며, 기본 실험은 SIFT1M(128d)으로 수행

추가 확보 시도 (미확정): - 연구실 대학원생으로부터 Exqutor 패치 빌드 환경 또는 실험 데이터셋을 제공받을 수 있는지 확인 중 - 확보 실패 시 자체 빌드 또는 ECQO 비교를 제외한 Baseline vs Cascade 비교로 축소

벡터 컬럼 추가 방법: TPC-H 테이블에 벡터 컬럼을 추가하여 VAQ 벤치마크를 구성한다. Exqutor GitHub의 [Vector-augmented_SQL_analytics/](#) 스크립트가 이 과정을 자동화하며, 자체 구성 시에는 SIFT1M 벡터를 TPC-H lineitem/part 테이블에 매핑한다.

4.3 실험 단계

1단계: 병목 프로파일링 (2주)

- 환경 구축 직후 **EXPLAIN** 으로 range query(**WHERE embedding <-> query < D**)가 HNSW 인덱스를 활용하는 지 최우선 확인. 활용 불가 시 IVFFlat 또는 Exqutor 패치 적용 후 재확인
- **EXPLAIN (ANALYZE, TIMING, BUFFERS, FORMAT JSON)** 으로 Planning/Execution 분리 측정
- 선택도 sweep: 0.01% ~ 90%, 8개 지점. 이 sweep 결과에서 선택도 ~33%에 대응하는 D 값을 역산하여 2단계 D_loose 파라미터 결정에 활용
- 필터 복잡도: 0개 ~ 3개+조인, 4단계
- 산출물: Planning vs Execution 비율 곡선, Q-error 곡선, 병목 영역 식별, D_loose 후보값

2단계: Cascaded Decomposition 실험 (2.5주)

- 사전 확인: PostgreSQL 16에서 비재귀 CTE의 자동 인라인 여부를 **EXPLAIN** 으로 검증. **MATERIALIZED** 키워드 유무에 따른 실행 계획 차이를 기록한 뒤 Phase 2b 캐싱 전략 설계에 반영
- Phase 2a (12~16개 조합): 분해 임계값 4가지 × 비교 전략 4가지(Baseline / ECQO only / Cascade only / ECQO + Cascade). 필터 없이 고정.
- Phase 2b (24~36개 조합): 유의미한 조합(latency 10%+ 차이)만 → 필터 4단계 × 캐싱 전략 3가지(CTE / Materialized CTE / Temp Table + Index)로 확장
- 측정: 각 조합 5회 실행, 첫 1회 워밍업 제외, 나머지 4회 중앙값
- 산출물: 분해 임계값 × end-to-end latency 최적점 그래프, ECQO vs Cascade 비교표

3단계: ECQO 상호작용 + 확장 실험 (1.5주, 택 1~2개)

- ECQO + Cascade 결합 시 추가 향상 여부 확인
- top-k 전환 (1순위, k=10/100/1000)
- pgvector 0.8.0 비교 (2순위)
- 산출물: ECQO 상호작용 분석, 확장 조건별 성능 비교 데이터

4.4 평가 지표

지표	설명
Planning Time	EXPLAIN ANALYZE의 플래닝 소요 시간
Execution Time	EXPLAIN ANALYZE의 실행 소요 시간
end-to-end latency	Planning + Execution 합계 (최종 판단 기준)
Q-error [2]	$\max(\text{estimated}/\text{actual}, \text{actual}/\text{estimated})$

5. 위험 요소 및 대비 계획

위험 요소	영향	대비 계획
Exqutor 패치 빌드 실패	ECQO 비교(RQ3) 불가	Baseline vs Cascade only로 축소. pg_hint_plan으로 oracle plan 간접 비교
대학원생 데이터 미확보	TPC-H VAQ 직접 구성 필요	SIFT1M + TPC-H SF1 자체 생성. 벡터-테이블 매핑 스크립트 자체 작성
Planning Time 미미 (< 10%)	RQ1 가설 방향 전환	"구조적 pre-filtering 효과"로 재프레이밍. 1단계 자체는 독립 기여
CTE 실체화 오버헤드 > 이점	Cascade 자체 비효율	Temp Table + Index 전략으로 전환, 또는 분해 임계값 간격 조정
SF1 데이터가 너무 작음	차이 관찰 어려움	조인 복잡도 높이기, 또는 SF10(서버 필요)으로 확대
range query가 HNSW 미지원	Stage 1 인덱스 스캔 불가	IVFFlat 인덱스 대체 또는 Exqutor 패치 적용 후 재확인

6. 팀원 역할

구분	이름	비고
팀원 1	박세은	팀장
팀원 2	강재현	핵심 아이디어 제안
팀원 3	조현빈	
팀원 4	이동욱	

단계별로 역할을 유동적으로 배분하며, 전원이 모든 단계에 참여한다.

7. 수행 일정

기간	마일스톤	산출물
3/31	연구 방향 확정 (팀 논의)	팀 논의 완료, 초안 피드백 반영
4/1	제안서/계획서 작성 마무리	최종본 완성
4/2	제안서 + 계획서 제출	제출 완료
4/1~4/14	환경 구축	PG16+pgvector+pg_hint_plan, TPC-H VAQ 데이터
4/14~4/28	1단계 실험	Planning vs Execution 비율 곡선, Q-error 곡선
4/28~5/15	2단계 실험	분해 임계값 최적점, ECQO 비교표, 캐싱 전략 비교
5/15~5/25	3단계 확장	ECQO 상호작용 + 확장 실험 (택 1~2개)
5월 말	중간발표	발표 자료, 중간보고서
6월	최종 정리	최종발표, 최종보고서, 전시회

8. 예상 산출물

1. Planning Time vs Execution Time 비율 프로파일 — VAQ 병목의 정량적 분석
2. 선택도 vs Q-error 곡선 — 기존 추정이 틀리는 영역 시각화
3. 분해 임계값 × end-to-end latency 최적점 그래프 — Cascade의 sweet spot
4. ECQO vs Cascade vs 결합 성능 비교표 — 보완/대체 관계 판별
5. 조건별 최적 전략 가이드라인 — 실무 적용 권장 사항

9. 참고문헌

- [1] H. Kim, C. Lim, H. An, R. Sen, and K. Park, "Exqutor: Extended Query Optimizer for Vector-Augmented Analytical Queries," arXiv:2512.09695v2, 2025.
- [2] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How Good Are Query Optimizers, Really?" *Proc. VLDB Endowment*, vol. 9, no. 3, pp. 204-215, 2015.
- [3] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, and M. Cilimdžić, "Robust Query Processing through Progressive Optimization," in *Proc. ACM SIGMOD*, 2004.